

MICROCONTROLLERS

Labo 2

Naslag

Gegenereerd op 29/07/2026 uit tdmts.github.io/Microcontrollers

Inhoud

Analoge in- en uitgangen

[analogRead\(\)](#)

[analogWrite\(\)](#)

[map\(\)](#)

[De spanningsdeler](#)

Datasheets

[TMP35 / TMP36 / TMP37 los document](#)

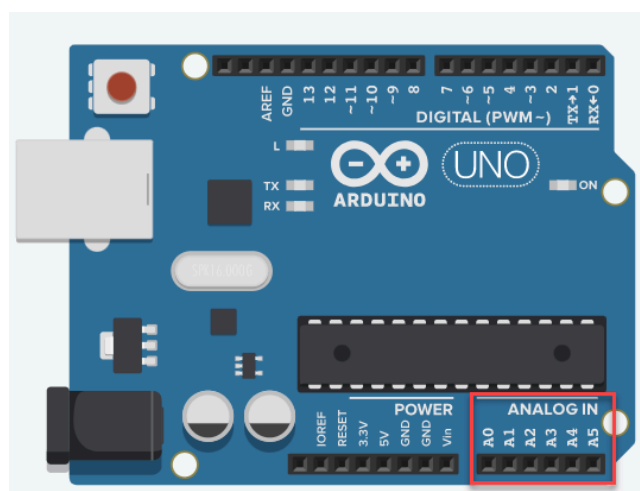
[LM35 los document](#)

[LDR 3190 los document](#)

NASLAG

analogRead

Met **analogRead()** kan je analoge spanningen inlezen op de analoge pinnen van je Arduino.



[Klik op de afbeelding om te vergroten](#)

Deze functie zet een analoge spanning om naar een digitale waarde die je in je programma kan gebruiken.

Hoe werkt het?

De Arduino gebruikt een ingebouwde ADC (Analog-to-Digital Converter) om analoge signalen om te zetten naar digitale waarden. Bij de meeste Arduino borden (zoals de Uno of Leonardo) heeft de ADC een resolutie van 10 bits, wat betekent dat de waarde tussen **0 en 1023** ($2^{10} = 1024$) ligt:

- 0 komt overeen met 0V
- 1023 komt overeen met de referentiespanning (5V bij de Uno en Leonardo)
- 512 komt ongeveer overeen met 2.5 V

Voorbeeld

CPP

```
1  const int analogePin = A0;
2
3  void setup()
4  {
5      pinMode(analogePin, INPUT); // OPTIONEEL
6
7      Serial.begin(9600);
8  }
9
10 void loop()
11 {
12     int waarde = analogRead(analogePin); // Een geheel getal tussen 0 en 1023
13     double spanning = waarde / 1023.0 * 5; // De overeenkomstige spanning tussen 0V en 5V
14
15     Serial.print(waarde);
16     Serial.print(" komt overeen met ");
17     Serial.print(spanning);
18     Serial.println(" V");
19
20     delay(500); // anders overspoel je de seriële monitor
21 }
```

Waarom delen we door 1023.0 en niet door 1023?

Als we zouden delen door 1023 (een int) dan is in de breuk zowel de teller als de noemer een int en dan maken we de gehele deling. Het resultaat zou dus geen cijfer na de komma bevatten. Zie

ook het hoofdstuk Wiskundige operatoren in labo 0.

analogWrite

Met `analogWrite()` kan je een analoge spanning **simuleren** op bepaalde pinnen van je Arduino. Dit noemen we **PWM** (Pulse Width Modulation).

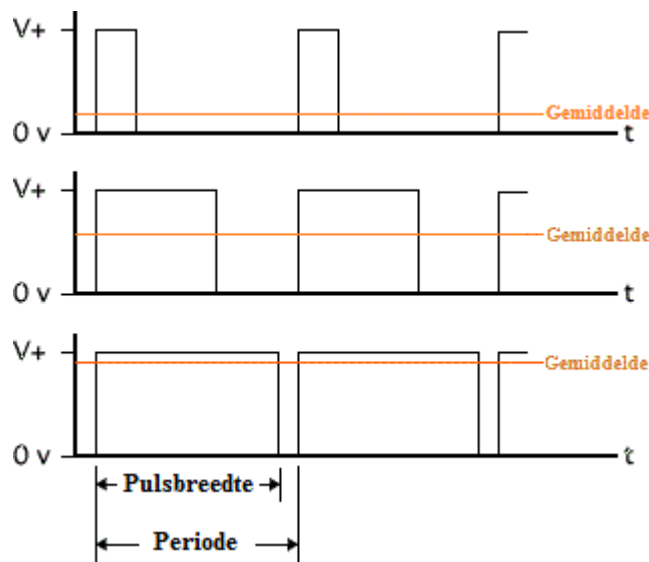
Hoe werkt het?

PWM werkt door een digitaal signaal heel snel aan en uit te zetten. Dit gebeurt ongeveer 490 keer per seconde (490 Hz) op een Arduino Uno of Leonardo. Door de verhouding tussen "aan tijd" en "uit tijd" te variëren, kan je bijvoorbeeld een led dimmen of de snelheid van een motor regelen.

Je geeft in de functie `analogWrite` een waarde mee tussen **0 en 255**:

- **0** = altijd uit (0% duty cycle, 0V gemiddeld)
- **127** = 50% van de tijd aan (50% duty cycle, ~2.5V gemiddeld)
- **255** = altijd aan (100% duty cycle, 5V gemiddeld)

De resolutie (het aantal mogelijke toestanden) is bij `analogWrite` slechts **256** (en niet 1024 zoals bij `analogRead` het geval is). Dit komt omdat hier 8 bits worden gebruikt ($2^8 = 256$) in plaats van 10 bits.



[Klik op de afbeelding om te vergroten](#)

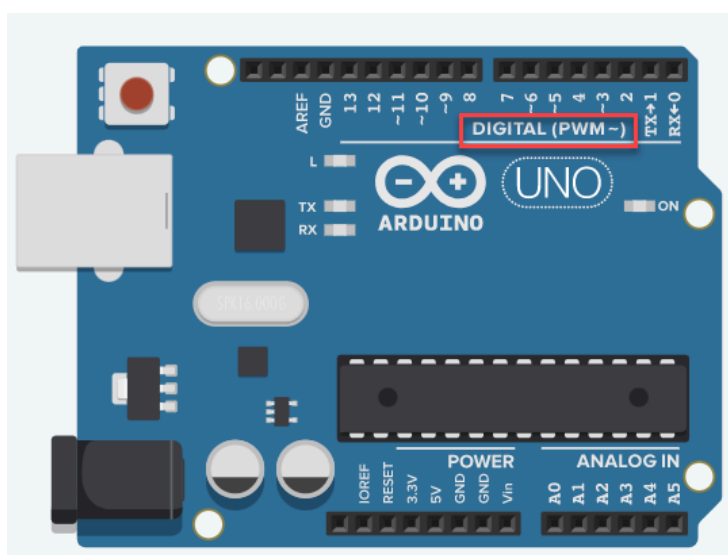
Voorbeeld

CPP

```
1  const int potPin = A0; // Potentiometer op analoge pin A0
2  const int ledPin = 3;  // LED op PWM pin 3
3
4  void setup()
5  {
6      pinMode(ledPin, OUTPUT);
7  }
8
9  void loop()
10 {
11     // Lees de potentiometer uit (0-1023)
12     int potWaarde = analogRead(potPin);
13
14     // Herschaal naar PWM waarde (0-255)
15     int helderheid = potWaarde / 1023.0 * 255;
16
17     // Stuur PWM signaal naar LED
18     analogWrite(ledPin, helderheid);
19 }
```

⚠ PWM WERKT ALLEEN OP DE PINNEN MET EEN ~

Op de Arduino Uno en Leonardo zijn dat de pinnen met een ~ symbool ernaast (pin 3, 5, 6, 9, 10, 11). De andere pinnen ondersteunen geen PWM.



[Klik op de afbeelding om te vergroten](#)

i WAAROM AUTOLICHTEN SOMS FLIKKEREN OP VIDEO

Achteraan een auto zit dezelfde lamp in twee rollen: zwak als achterlicht, fel als remlicht. Dat verschil in helderheid maakt de fabrikant met PWM. Je oog volgt dat schakelen niet, maar een camera neemt beeld voor beeld op en valt daarbij soms in een uit-periode, waardoor je op de opname strepen of geflikker ziet die er in het echt niet zijn.



📄 Elk van die achterlichten wordt met PWM op zijn helderheid gehouden. Bron: StockCake

map()

Met **map()** kan je een waarde van het ene bereik omzetten naar een ander bereik. Dit is handig wanneer je bijvoorbeeld een sensorwaarde (0-1023) wil omzetten naar een PWM-waarde (0-255), of wanneer je een percentage wil berekenen.

Hoe werkt het?

```
long result = map(value, fromLow, fromHigh, toLow, toHigh);
```

CPP

- **value**: de waarde die je wil omzetten
- **fromLow**: ondergrens van het invoerbereik
- **fromHigh**: bovengrens van het invoerbereik
- **toLow**: ondergrens van het uitvoerbereik
- **toHigh**: bovengrens van het uitvoerbereik

Voorbeeld 1

Lineaire analogRead naar analogWrite

```
1  const int potPin = A0;
2  const int ledPin = 3;
3
4  void setup()
5  {
6      pinMode(ledPin, OUTPUT);
7  }
8
9  void loop()
10 {
11     // Lees de potentiometer uit (0-1023)
12     int potWaarde = analogRead(potPin);
13
14     // Map van 0-1023 naar 0-255
15     long helderheid = map(potWaarde, 0, 1023, 0, 255);
16
17     // Stuur PWM signaal naar LED
18     analogWrite(ledPin, helderheid);
19 }
```

Voorbeeld 2

Geïnverteerde lineaire analogRead naar analogWrite

```
1  const int potPin = A0;
2  const int ledPin = 3;
3
4  void setup()
5  {
6      pinMode(ledPin, OUTPUT);
7  }
8
9  void loop()
10 {
11     // Lees de potentiometer uit (0-1023)
12     int potWaarde = analogRead(potPin);
13
14     // Map van 0-1023 naar 0-255
15     long helderheid = map(potWaarde, 0, 1023, 255, 0);
16
17     // Stuur PWM signaal naar LED
18     analogWrite(ledPin, helderheid);
19 }
```

Voorbeeld 3

Dead zone potentiometer waarbij we de eerste 10% de led volledig uitzetten en de laatste 10% de led volledig aanzetten.

CPP

```
1  const int potPin = A0;
2  const int ledPin = 3;
3
4  void setup()
5  {
6      pinMode(ledPin, OUTPUT);
7  }
8
9  void loop()
10 {
11     // Lees de potentiometer uit (0-1023)
12     int potWaarde = analogRead(potPin);
13     long helderheid = 0;
14
15     if(potWaarde < 102)
16     {
17         helderheid = 0;
18     }
19     else if(potWaarde > 922)
20     {
21         helderheid = 255;
22     }
23     else
24     {
25         // Map van 10%-90% naar 0%-100%
26         helderheid = map(potWaarde, 102, 922, 0, 255);
27     }
28
29     // Stuur PWM signaal naar LED
30     analogWrite(ledPin, helderheid);
31 }
```

De spanningsdeler

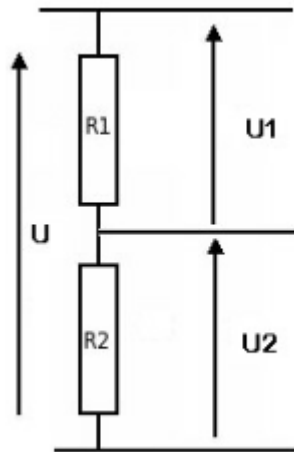
Twee weerstanden in serie verdelen de spanning onder elkaar. Zo lees je zowat elke weerstandssensor uit: een LDR, een thermistor, een rekstrookje. Je Arduino kan namelijk geen weerstand meten, alleen spanning.

Waarom je dit nodig hebt

Een LDR is een weerstand die verandert met het licht: donker geeft veel weerstand, licht weinig. Maar `analogRead()` meet geen ohm, het meet volt. Hang je de LDR rechtstreeks tussen 5 V en een analoge ingang, dan meet je altijd 5 V, hoe donker het ook is. Er loopt immers geen stroom, dus valt er ook geen spanning over de LDR.

Je hebt dus een tweede weerstand nodig, zodat er wel stroom kan lopen en de spanning zich over de twee kan verdelen. Die verdeling verschuift mee met je sensor, en dat is wat je meet.

De formule



[🔗](#) Klik op de afbeelding om te vergroten

Twee weerstanden in serie over een spanning U verdelen die spanning in verhouding tot hun waarde. De spanning over $R2$ is:

$$U_2 = U * \frac{R_2}{R_1 + R_2}$$

 *Klik op de afbeelding om te vergroten*

Je meet met je Arduino het punt **tussen** de twee weerstanden, dus U_2 . Dat volgt rechtstreeks uit [de wet van Ohm](#): door beide weerstanden loopt dezelfde stroom, dus verhouden hun spanningen zich als hun weerstandswaarden.

DRIE GRENSGEVALLEN

Is R_2 veel groter dan R_1 , dan gaat bijna de hele spanning naar R_2 en meet je ongeveer 5 V. Is R_2 veel kleiner, dan meet je bijna 0 V. Zijn ze gelijk, dan meet je precies de helft. Meer dan die drie punten heb je meestal niet nodig om te weten of je schakeling de goede kant opgaat.

De vaste weerstand kiezen

Bij een sensor is één van de twee weerstanden de sensor zelf, en de andere kies je. Die keuze bepaalt hoe bruikbaar je meting wordt.

Neem je een vaste weerstand die veel te groot of veel te klein is tegenover je sensor, dan zit je meting altijd tegen 5 V of tegen 0 V geplakt en verandert er nauwelijks iets als de sensor beweegt. Je gooit dan het grootste deel van je 1024 meetstappen weg.

KIES ROND HET MIDDEN VAN JE SENSORBEREIK

Kies je vaste weerstand ergens in het **midden van het bereik** dat je sensor doorloopt, meetkundig gezien. Varieert je LDR tussen 400 Ω en 9000 Ω , dan zit je goed rond de 2 k Ω . Zo krijg je het grootste verschil tussen je laagste en je hoogste meting.

Twijfel je tussen een paar waarden die je in huis hebt, vul dan de formule in voor elk ervan en kijk welke het grootste verschil tussen minimum en maximum oplevert. Dat is wat je bij [LDR: lichtmeting](#) doet.

De potentiometer is er ook een

Een potentiometer met drie pootjes is een spanningsdeler waarvan je het middenpunt kan verschuiven. De twee buitenste pootjes hang je aan 5 V en aan GND, en het middelste pootje, de loper, is precies dat punt tussen R_1 en R_2 . Draaien aan de knop verschuift de verhouding tussen de twee helften, en dus de spanning die je meet.

Daarom levert een potentiometer de volle 0 tot 1023 op. De looper kan van het ene uiterste naar het andere, dus van 0 V tot 5 V.

 BELAST JE DELER NIET ZWAAR

De formule hierboven gaat ervan uit dat er verder niets aan het middenpunt hangt dat stroom trekt. Een analoge ingang van de Arduino voldoet daaraan: die is [hoogimpedant](#) en trekt vrijwel geen stroom. Hang je er wel iets aan dat stroom vraagt, bijvoorbeeld rechtstreeks een led, dan klopt je berekening niet meer en zakt de gemeten spanning in.